

Final Project: Milestone 3

Project Timeline (Tentative)

Milestone	Due Date	Deliverable
1 — Proposal	Apr 13	project proposal
2 — Design	Apr 27	detailed project description and software design
3 — GPU Kernels	May 11	Code + progress report on CUDA implementation
4 — Distributed	May 27	Code + report with MPI implementation, benchmarks
Final Report	Jun 8	Code + 6-page final report

The Final Report deadline will not be changed. Other deadlines are tentative at this point.

Milestone 3 Instructions

Overview

The goal of Milestone 3 is to implement and optimize the computational GPU kernels at the core of your project. By the end of this milestone, you should have a working single-GPU implementation of the main algorithms identified in your Milestone 2 design, together with preliminary performance measurements.

Milestone 4 will focus on scaling your implementation across multiple GPUs using MPI. For this milestone, you should concentrate on the correctness and performance of your CUDA kernels on a single GPU, along with any supporting CPU code needed to run and validate them.

Deliverables

You will turn in two items:

1. **Code.** The CUDA kernels implementing the computationally intensive parts of your algorithm, together with the C++ wrapper code (host-side driver, memory management, I/O, and any CPU reference or baseline code) needed to build, run, and test them.
2. **Progress report (approximately 2 pages, excluding references and figures).** A written description of your accomplishments since Milestone 2, your current status, and a preliminary performance analysis.

Report Content

Your progress report should address the following:

1. **Summary of progress.** Briefly describe what you have implemented so far. Which components of your Milestone 2 design are complete, which are partially done, and which are not yet started? Note any changes to your design and explain why.
2. **CUDA kernels.** Describe the GPU kernels you have implemented. For each significant kernel, explain what it computes, the parallelization strategy (thread and block organization), and the GPU features you are exploiting (e.g., shared memory, coalesced accesses, tensor cores). Mention any optimizations you have already applied and any you plan to apply later.
3. **C++ wrapper and CPU code.** Describe how your host code drives the GPU kernels: memory allocation and transfers, kernel launches, and any CPU-side computation or validation. If you have a CPU reference or baseline implementation, describe it as well.
4. **Correctness testing.** Describe how you verify that your kernels produce correct results. This may include comparison against a CPU reference, known analytical solutions, or published results. Report the accuracy achieved.
5. **Preliminary performance analysis.** Report initial performance measurements for your CUDA implementation, and for your CPU code where appropriate. Useful metrics include kernel runtime, achieved memory bandwidth, achieved compute throughput (GFLOPS), and speedup over your baseline. Include at least one figure or table summarizing your results. Comment on what the numbers suggest: is the kernel compute-bound, memory-bound, or latency-bound? How far are you from the theoretical peak of the hardware?

Profiling tools such as Nsight Compute or Nsight Systems are encouraged but not required at this stage.
6. **Challenges and next steps.** Describe the main technical difficulties you encountered and how you resolved them (or plan to). List the specific tasks remaining for Milestone 4 and the final report, with an updated timeline.

Submission

Submit your code and your progress report on Gradescope as a group. The report should be approximately 2 pages (excluding references and figures), single-spaced, 11pt font, 1-inch margins. Include all group members in the submission.